EXPLORE EGYPT | DEPI PROJECT

SYSTEM ANALYSIS & DESIGN

Discover Egypt Beyond the Famous Places

Submission document | Version 1.0

Prepared: 6 November 2026

Design baseline for team and instructor review

1. Executive Summary, Problem & Scope

Explore Egypt is a managed tourism discovery and trip-planning marketplace for Egyptian destinations, including lesser-known places. Visitors discover destinations and tours; registered travelers submit booking requests and track status; approved companies manage tours and departures; administrators manage content and company approval.

The proposed solution uses a layered web architecture: browser-based frontend, ASP.NET Core Web API, application/domain services, Entity Framework Core and SQL Server. The optional AI planner retrieves candidate places from the platform database before using an external provider.

Problem: Destination discovery, practical visit information, tour comparison and itinerary planning may require travelers to consult multiple sources. Explore Egypt brings these activities into one searchable platform.

Objectives: searchable Egyptian destinations; verified place details; approved company tours and schedules; traceable booking requests; database-grounded itinerary suggestions; maintainable and testable academic MVP.

In scope for MVP	Out of scope for MVP
Public place catalogue, search/filters and details	Online payments, refunds, commissions and payouts
Auth/roles, company review, tours, ordered stops, departures	Mobile native app, live chat, third-party booking engines
Booking requests and company decisions; no payments	Guaranteed live prices/availability, traffic/weather prediction
Basic admin content management; optional planner	Production marketplace operations

Design status and important assumptions

- Backend and database are proposed technology choices; frontend framework remains open until confirmed.
- The V2 ERD is adopted as the design baseline. Booking references a specific Departure.
- Pending requests do not reserve seats in the proposed policy; capacity is checked atomically at company confirmation.
- Deployment topology is a proposal, not proof that hosting has been completed.

2. Actors, Requirements & Architecture

Actor	Main capabilities	Access boundary
Visitor	Browse/search places, view tours and place details	Public published data
Registered Traveler	Submit booking, view own status, favorites/reviews/itineraries	Own private records
Company Representative	Manage assigned company, tours, departures and requests	Own company only; publish after approval
Administrator	Manage places/categories, approve companies, moderate content	Protected admin actions
External AI Provider	Optional itinerary formatting	Backend only; scoped candidate data

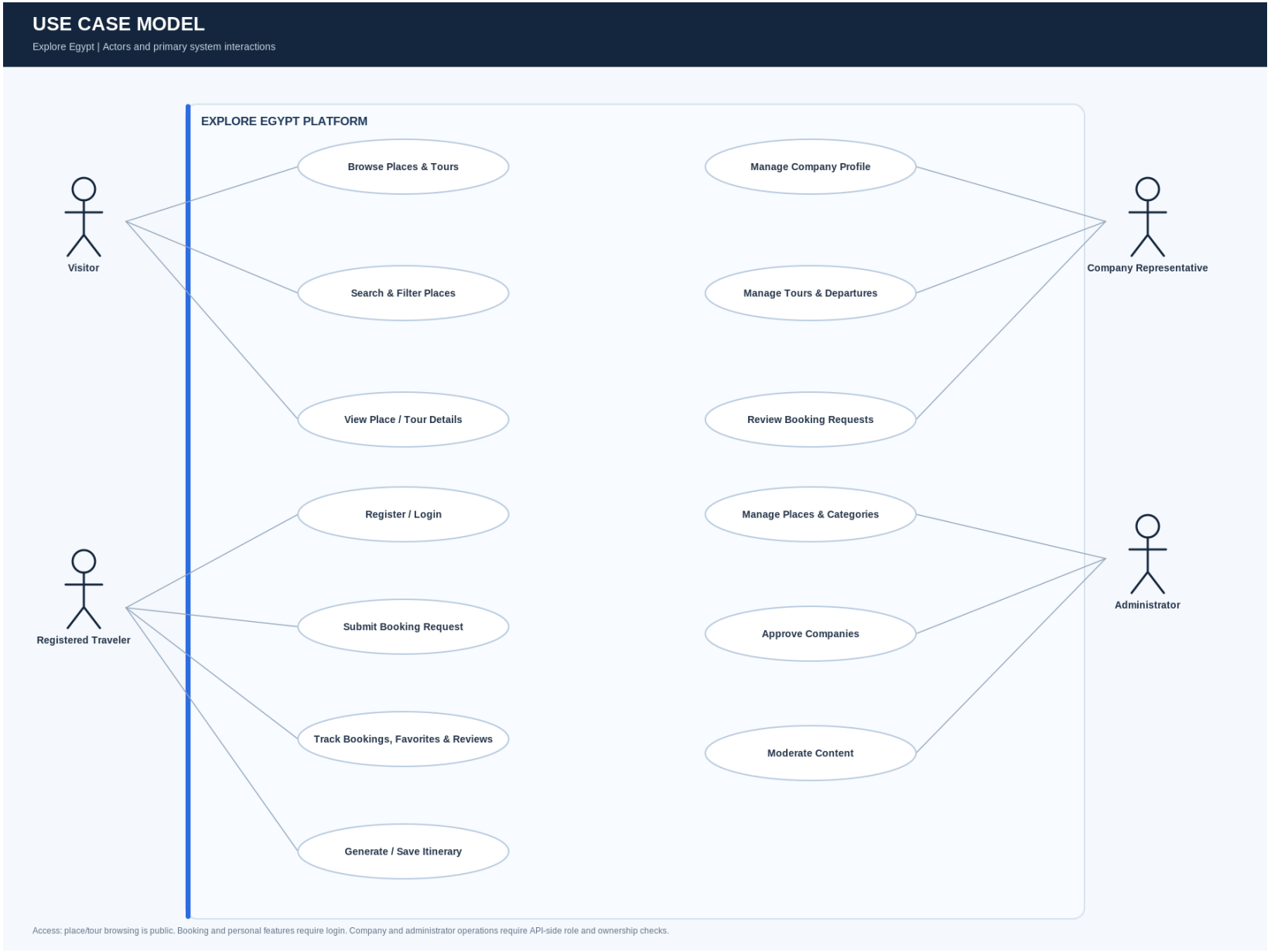
Architecture style: layered modular monolith

- Presentation: public website and traveler/company/admin dashboards.
- Application/API: endpoints, DTOs, validation, authentication, authorization and services.
- Domain: entities and business rules for places, tours, departures, bookings and itineraries.
- Persistence: EF Core mappings, migrations, SQL Server, indexes and transactions.
- External adapter: optional AI provider integration with timeout and fallback.

Proposed technology stack

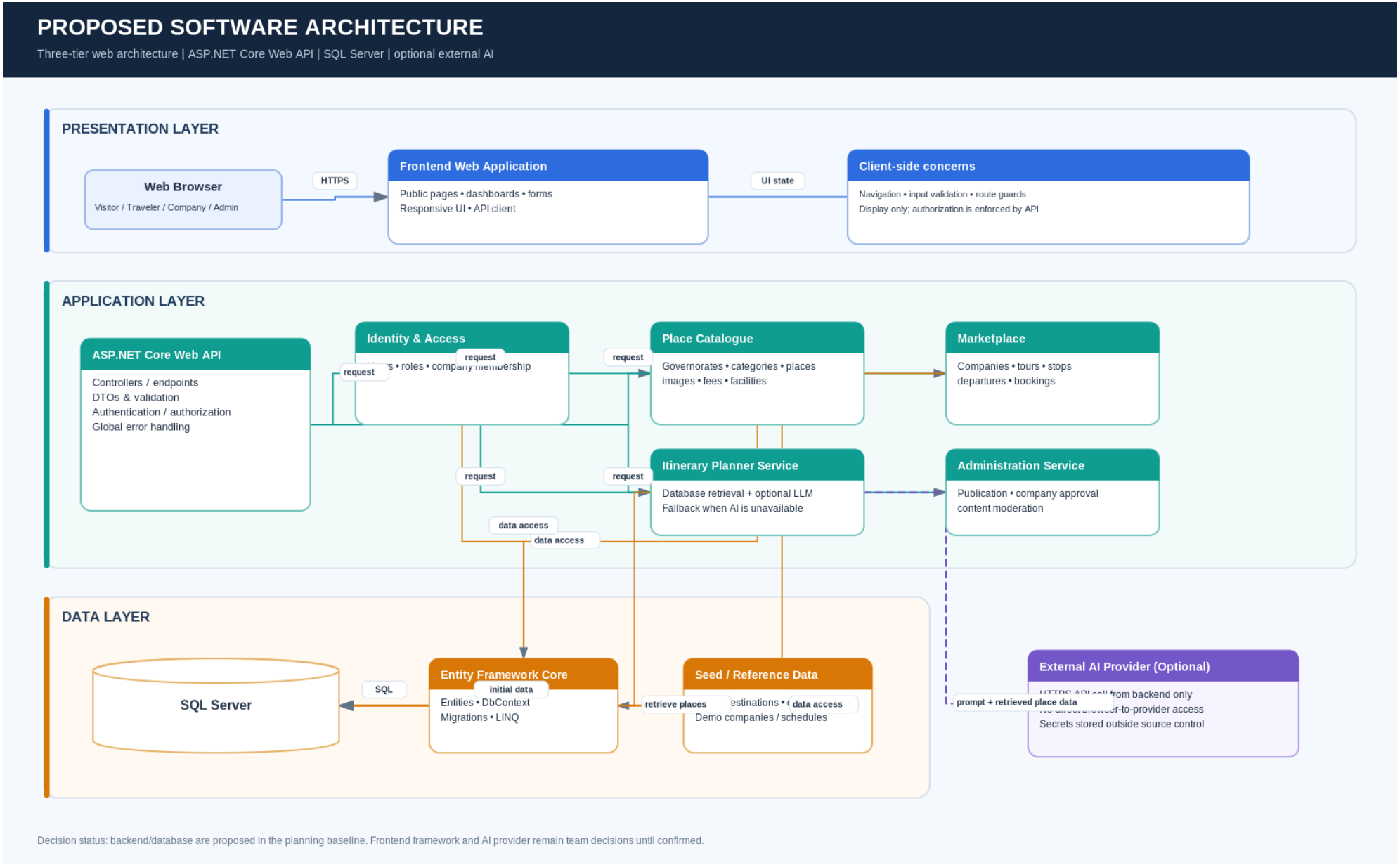
Area	Choice	Status
Backend	ASP.NET Core Web API, C#	Proposed baseline
ORM / Database	Entity Framework Core / SQL Server	Proposed baseline
API docs	Swagger / OpenAPI	Proposed baseline
Frontend	Angular OR HTML/CSS/JavaScript	Team decision open
AI	External LLM through backend adapter	Provider not selected; optional

3. Use Case Diagram



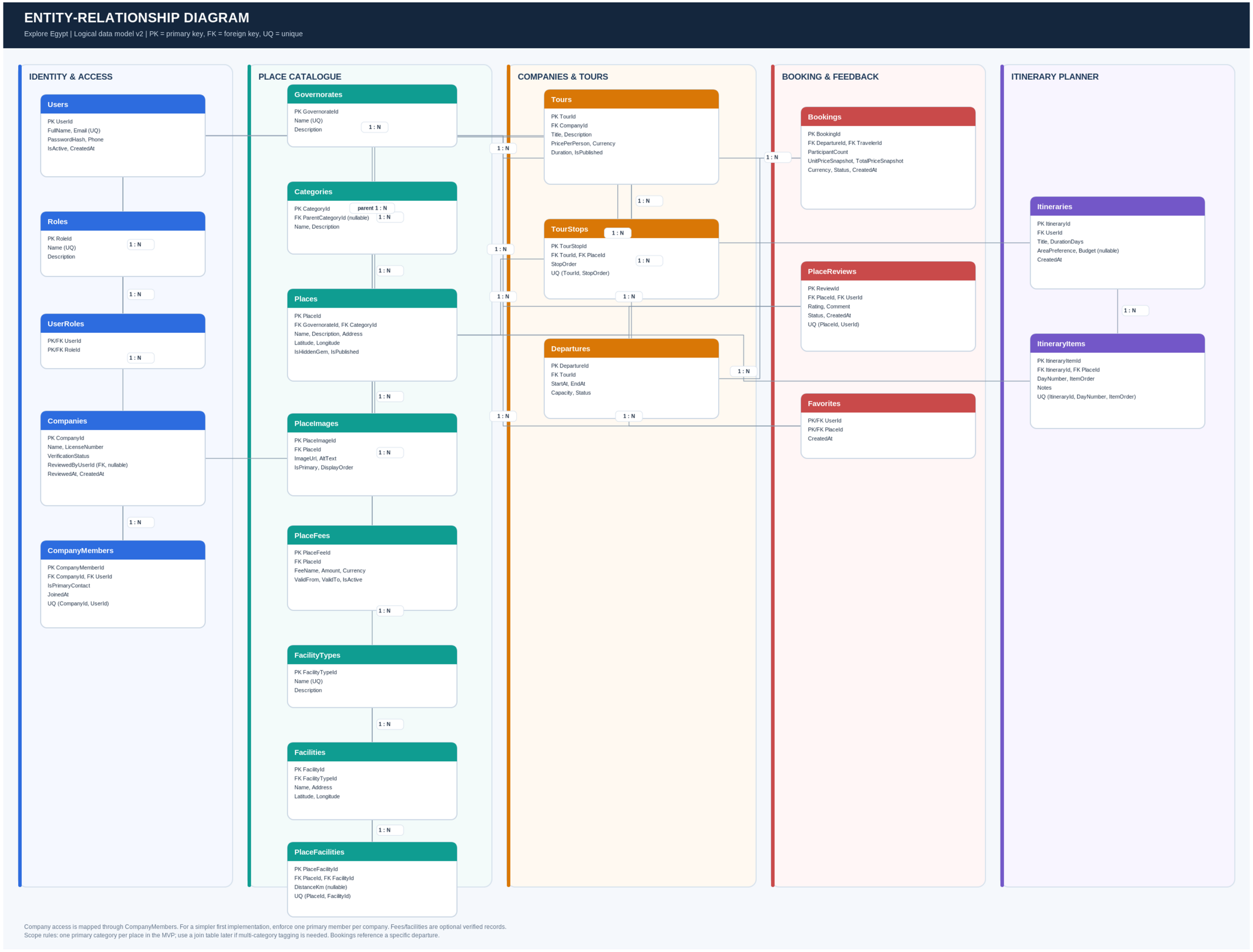
Public browsing is open; booking and personal features require authentication; company and admin operations are protected.

4. Software Architecture



Layered modular monolith with optional backend-only AI provider integration.

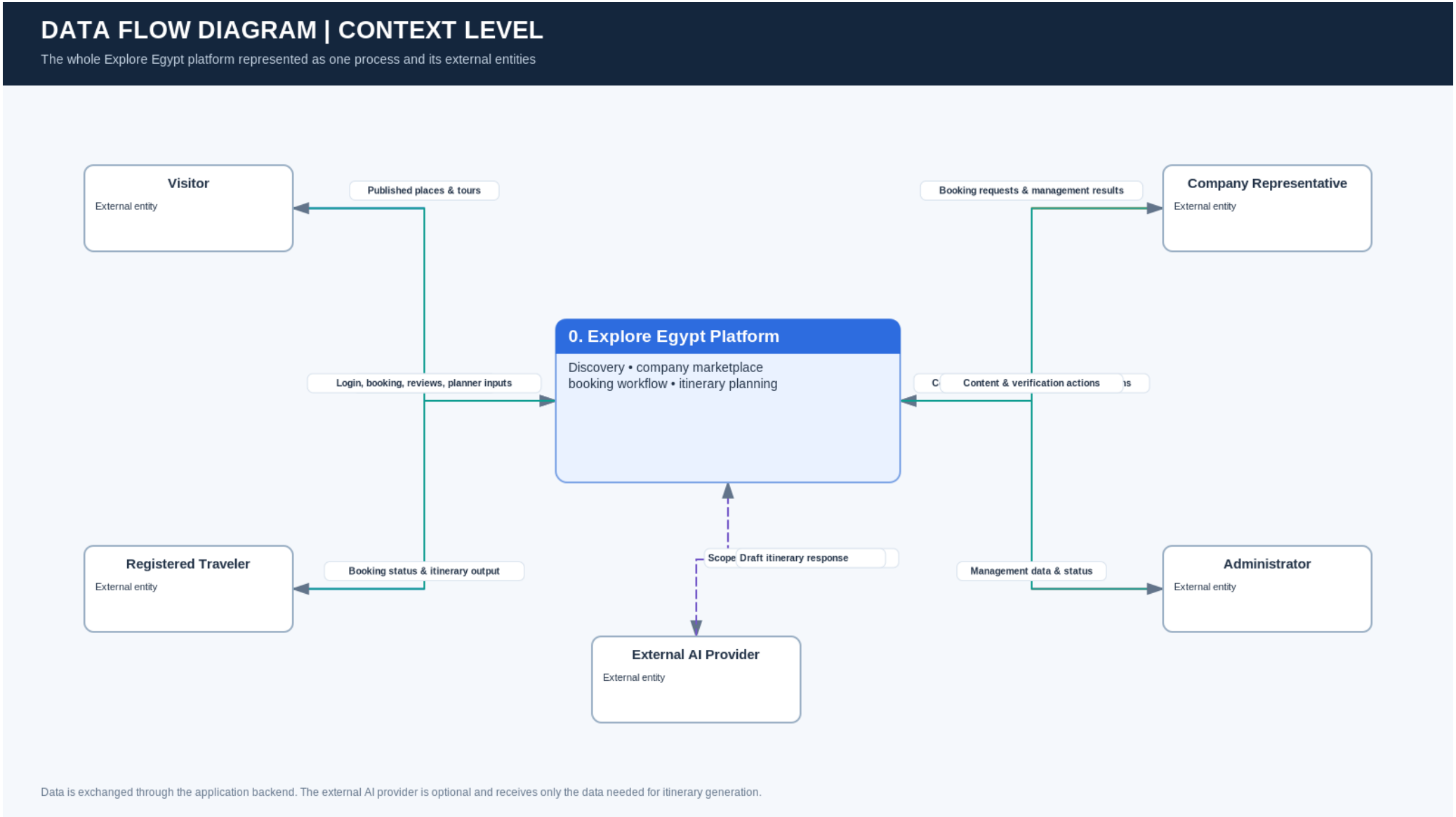
5. Entity-Relationship Diagram - V2



Company access is mapped through CompanyMembers. For a simpler first implementation, enforce one primary member per company. Fees/facilities are optional verified records.
Scope rules: one primary category per place in the MVP; use a join table later if multi-category tagging is needed. Bookings reference a specific departure.

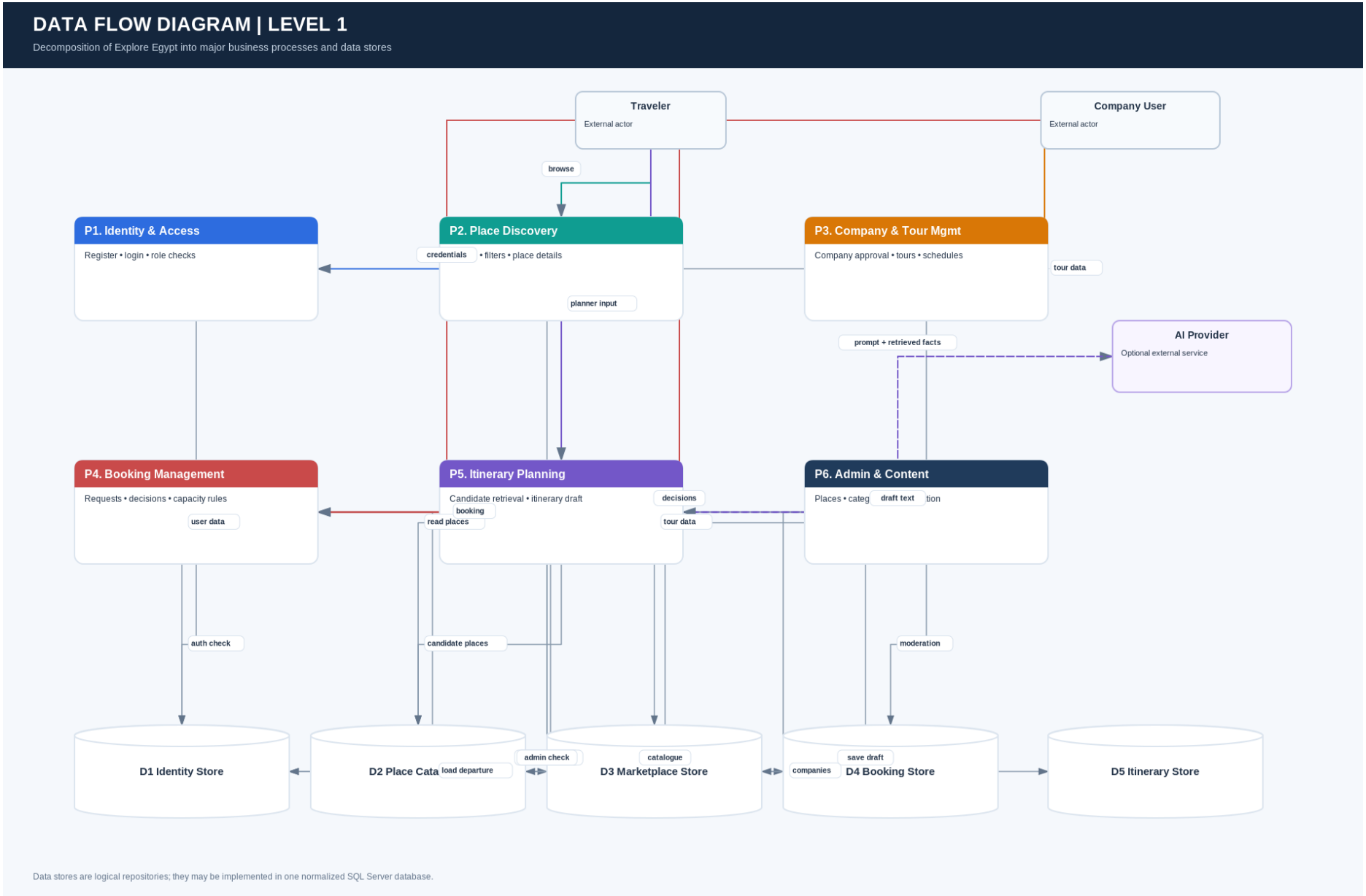
Logical ERD across identity, catalogue, marketplace, bookings/feedback and itinerary planning.

6. DFD - Context Level



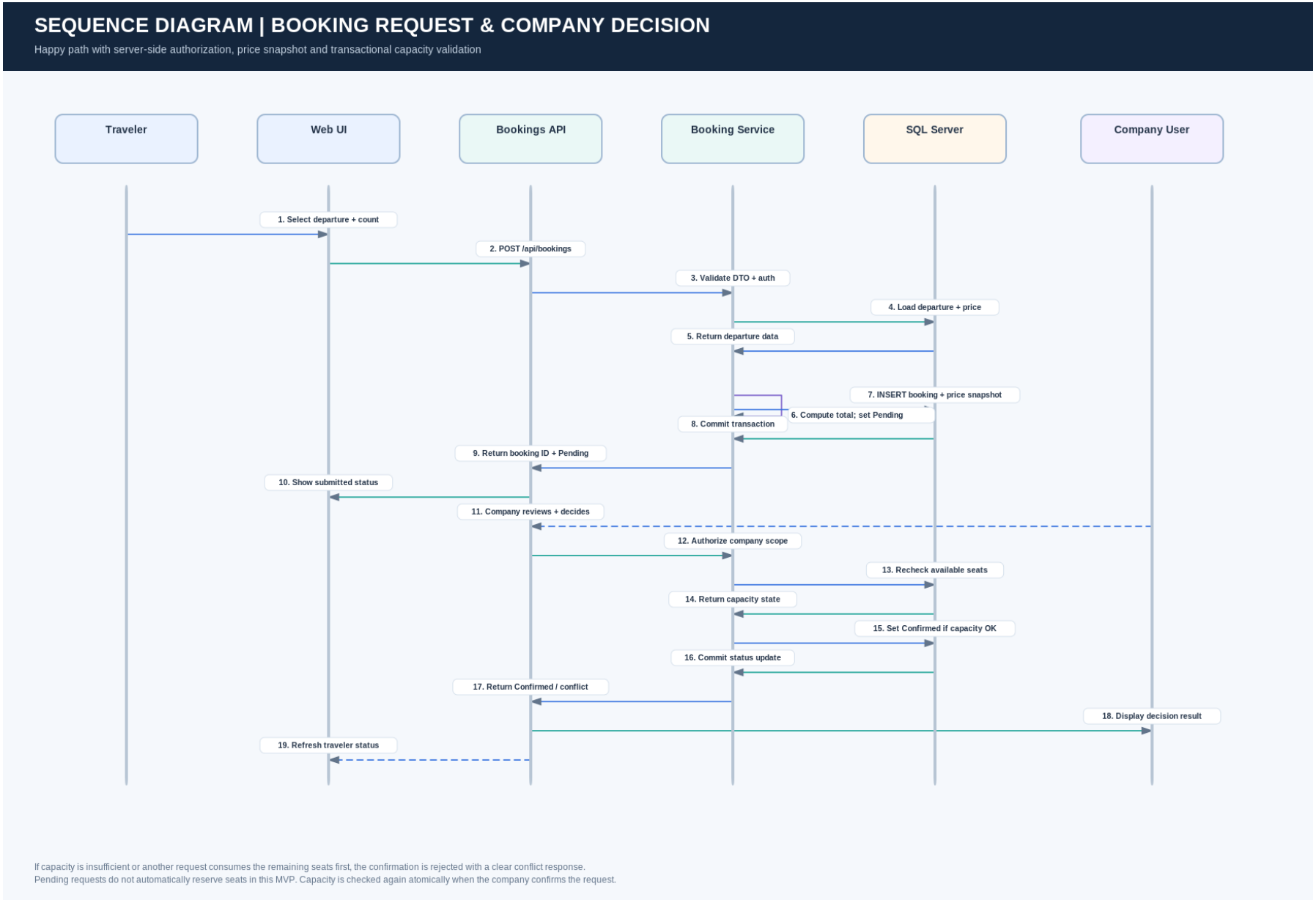
External actors exchange information with the platform through its backend.

7. DFD - Level 1



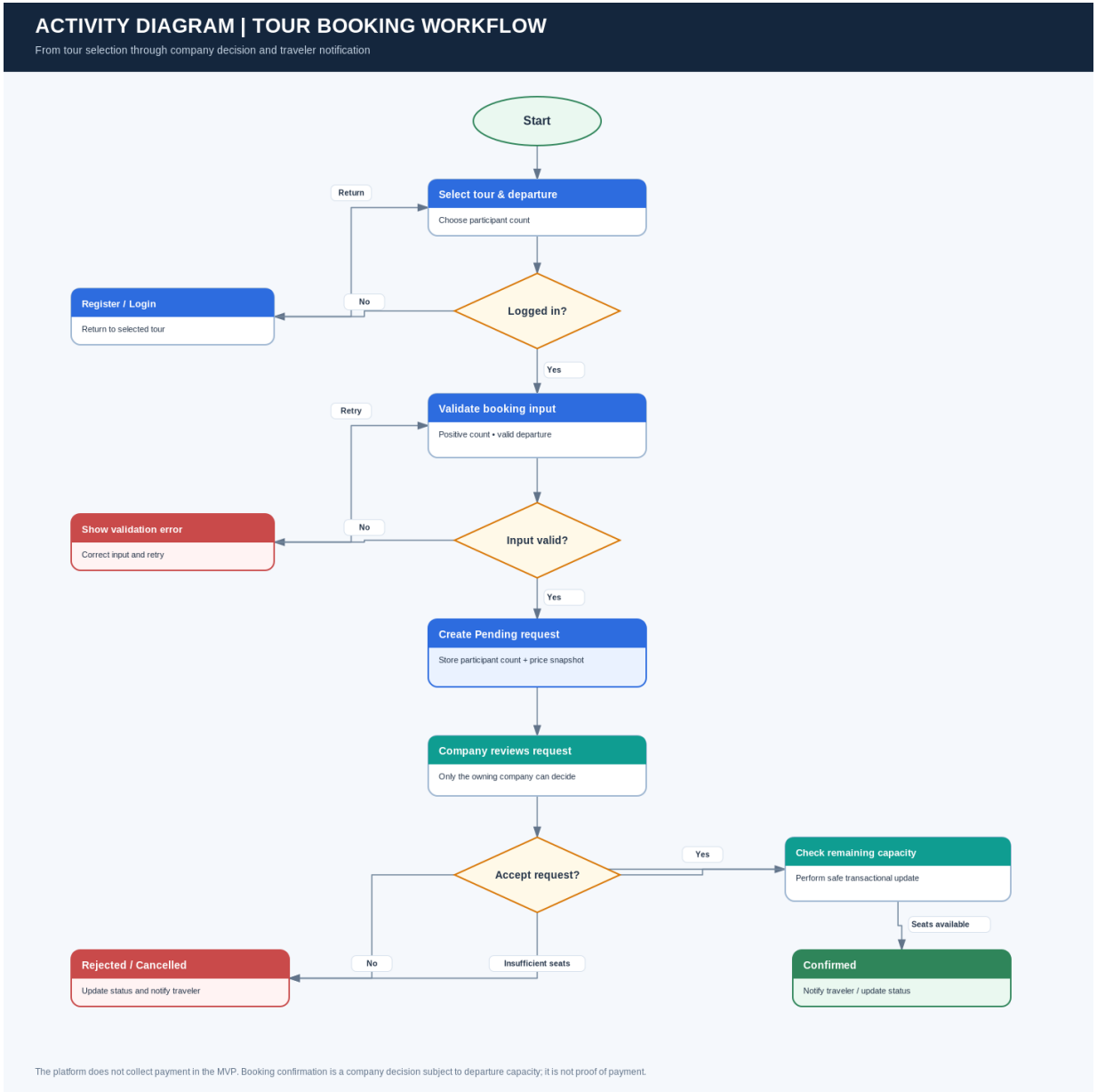
Major business processes and logical data stores; these can share one SQL Server database.

8. Sequence Diagram - Booking



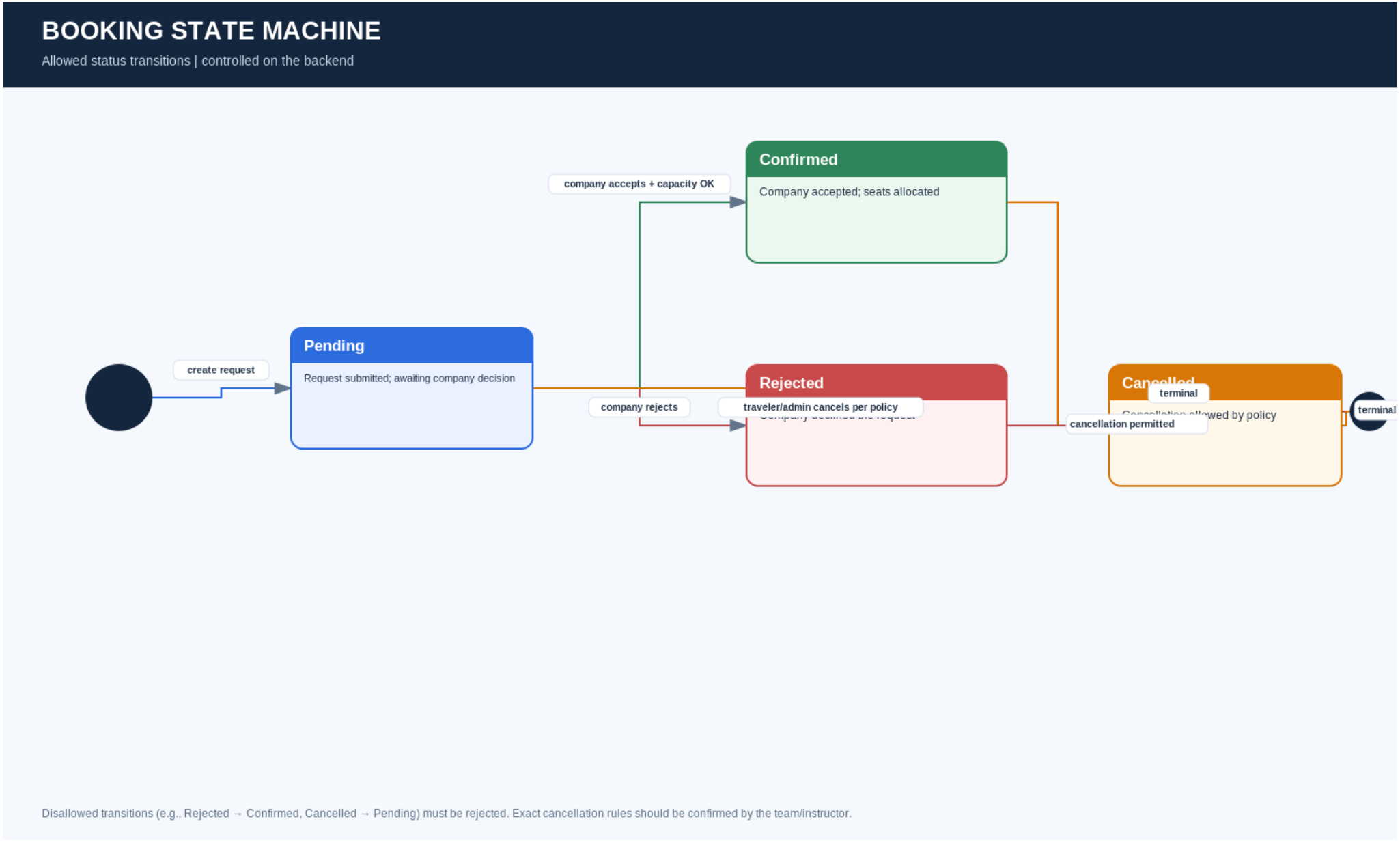
Submission, price snapshot, company decision and transactional capacity check.

9. Activity Diagram - Booking



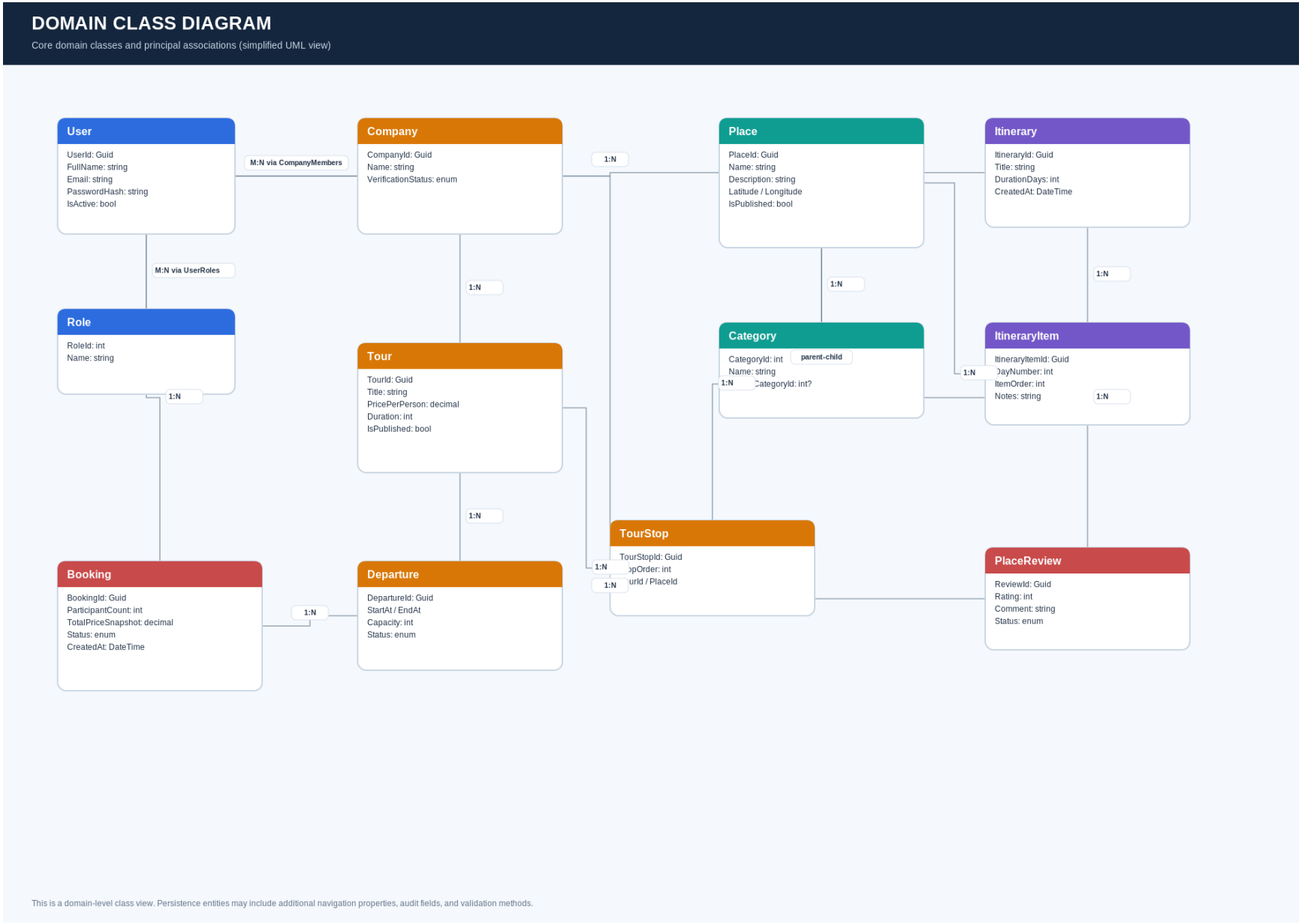
Authentication, validation, Pending creation, company review and capacity decision.

10. Booking State Machine



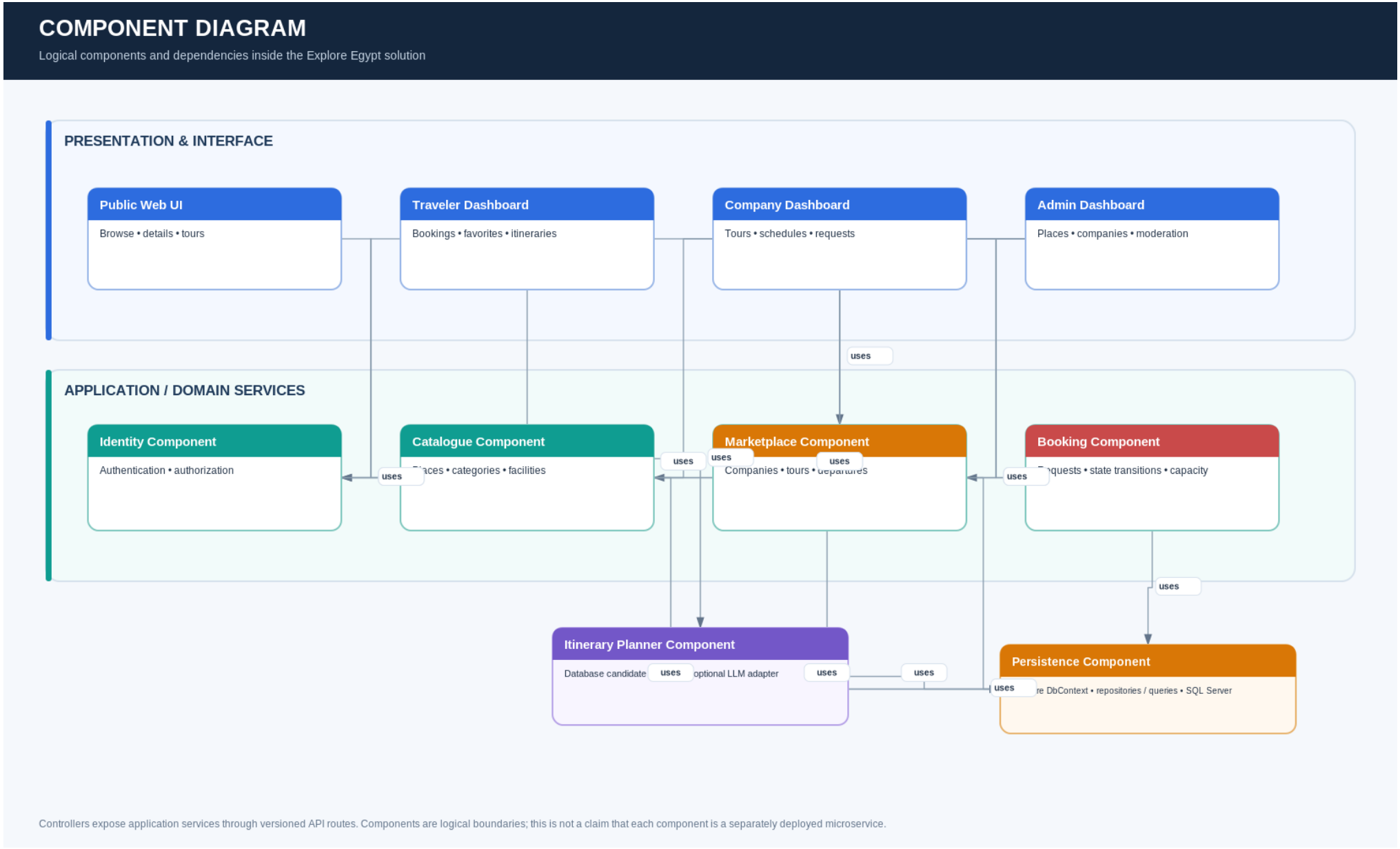
Allowed booking transitions; cancellation policy needs team confirmation.

11. Domain Class Diagram



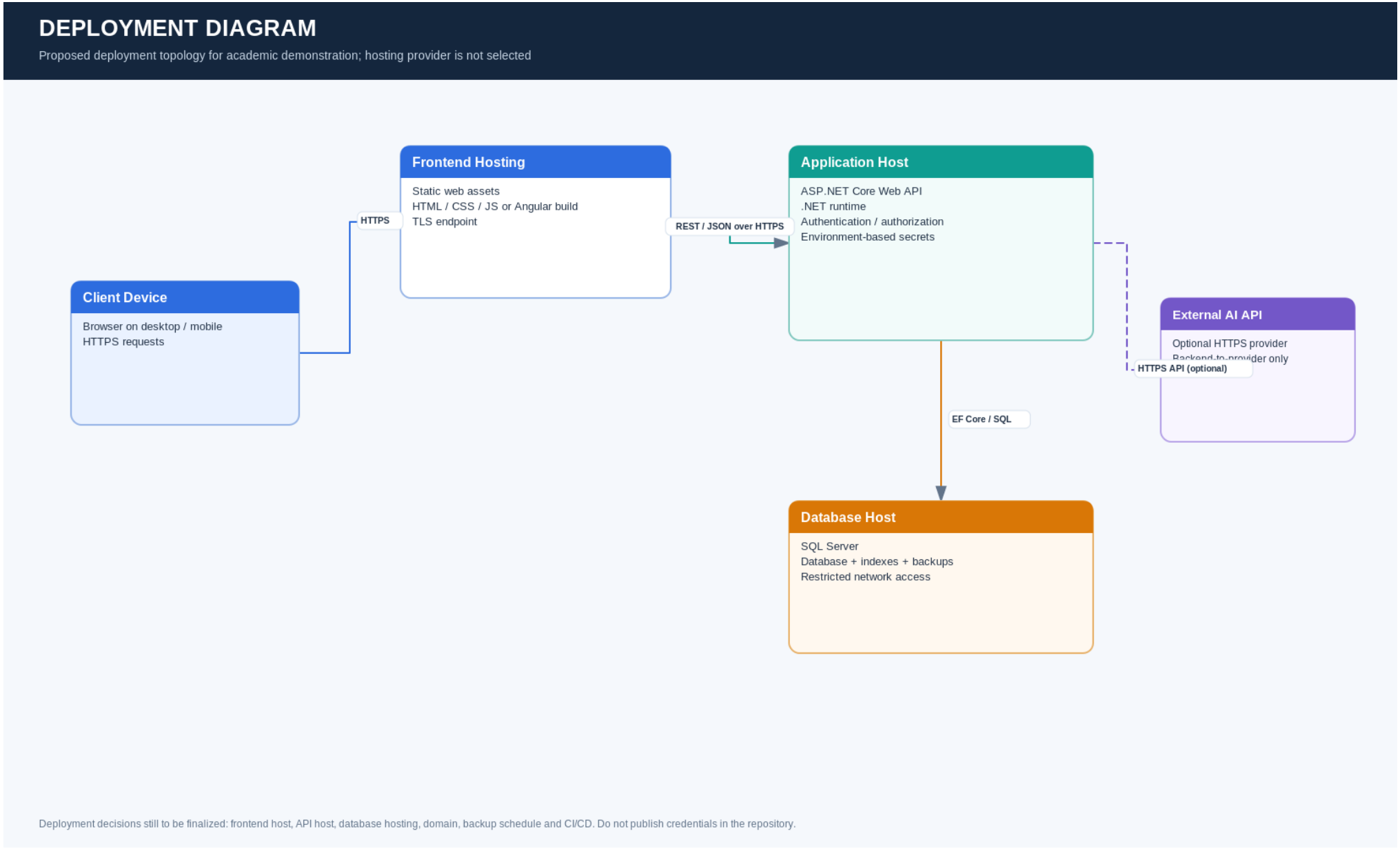
Simplified domain classes and key relationships.

12. Component Diagram



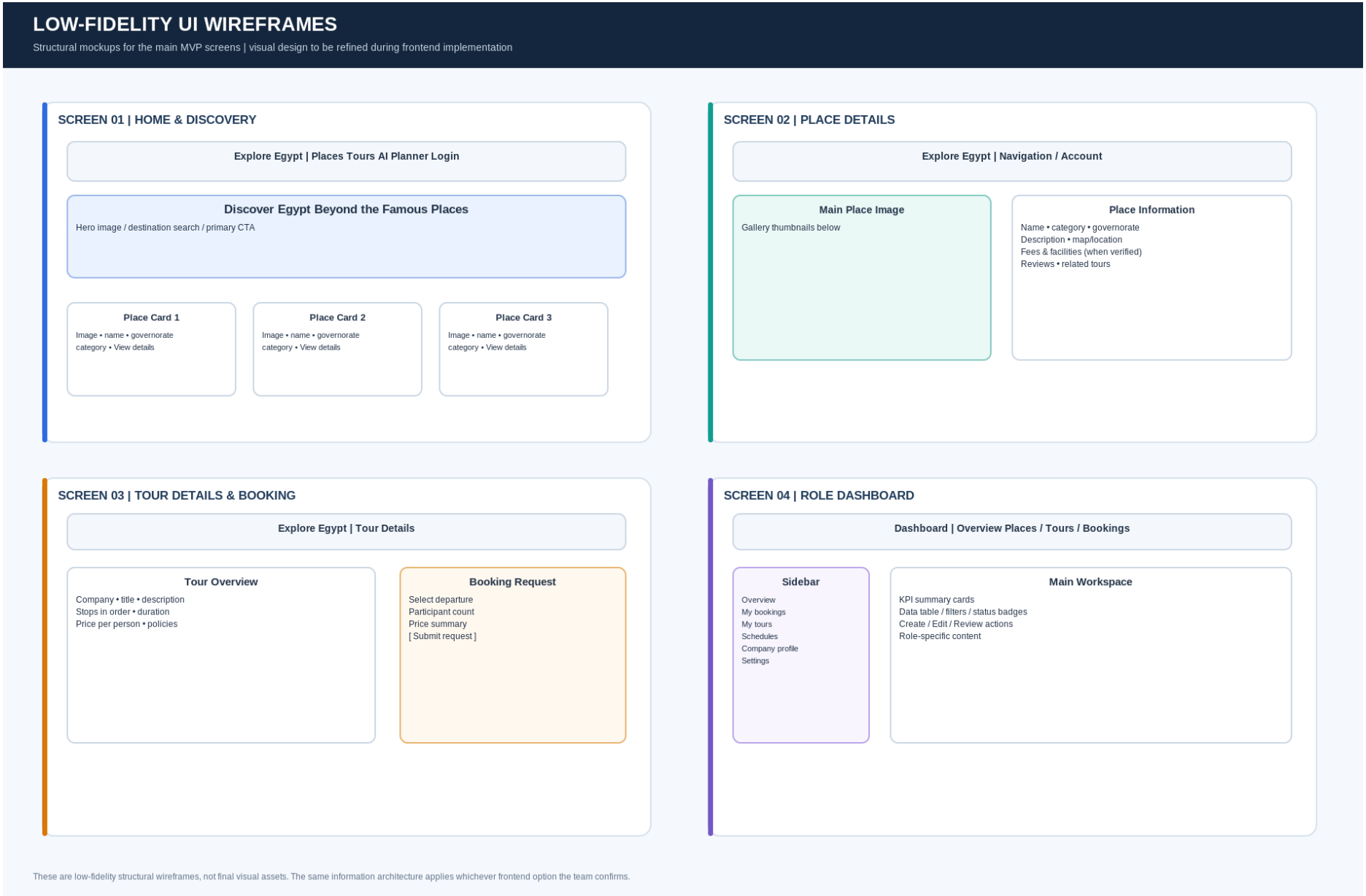
Logical modules and shared persistence boundary in the proposed modular monolith.

13. Deployment Diagram



Proposed runtime topology; deployment hosts are not selected yet.

14. UI/UX Wireframes



Low-fidelity structure for home, place details, tour booking and role dashboard.

15. Logical Schema - Entities & Constraints (1/2)

Table	Primary key	Important attributes / relationships	Key rules
Users	UserId	FullName, Email, PasswordHash, Phone, IsActive, CreatedAt	Email unique; password hash only
Roles	RoleId	Name, Description	Name unique
UserRoles	UserId + RoleId	FK to Users and Roles	Composite primary key
Companies	CompanyId	Name, LicenseNumber, VerificationStatus, ReviewedByUserId, ReviewedAt	Constrained verification status
CompanyMembers	CompanyMemberId	CompanyId, UserId, IsPrimaryContact, JoinedAt	Unique CompanyId + UserId
Governorates	GovernorateId	Name, Description	Name unique
Categories	CategoryId	ParentCategoryId nullable, Name, Description	Self-FK; no self-parent
Places	PlaceId	GovernorateId, CategoryId, Name, Description, Address, Lat/Long, IsHiddenGem, IsPublished	Valid FKs; coordinates in range
PlaceImages	PlaceImageId	PlaceId, ImageUrl, AltText, IsPrimary, DisplayOrder	Child records of Places
PlaceFees	PlaceFeeId	PlaceId, FeeName, Amount, Currency, ValidFrom/To, IsActive	Amount >= 0; valid date range
FacilityTypes	FacilityTypeId	Name, Description	Name unique
Facilities	FacilityId	FacilityTypeId, Name, Address, Lat/Long	Valid facility type FK
PlaceFacilities	PlaceFacilityId	PlaceId, FacilityId, DistanceKm nullable	Unique PlaceId + FacilityId

16. Logical Schema - Entities & Constraints (2/2)

Table	Primary key	Important attributes / relationships	Key rules
Tours	TourId	CompanyId, Title, Description, PricePerPerson, Currency, Duration, IsPublished	Price >= 0; company FK
TourStops	TourStopId	TourId, PlaceId, StopOrder	Unique TourId + StopOrder
Departures	DepartureId	TourId, StartAt, EndAt, Capacity, Status	EndAt > StartAt; capacity > 0
Bookings	BookingId	DepartureId, TravelerId, ParticipantCount, Unit/TotalPriceSnapshot, Currency, Status	Positive count; valid state; historical price snapshot
PlaceReviews	ReviewId	PlaceId, UserId, Rating, Comment, Status, CreatedAt	Rating range; optional unique PlaceId + UserId
Favorites	UserId + PlaceId	UserId, PlaceId, CreatedAt	Composite PK prevents duplicates
Itineraries	ItineraryId	UserId, Title, DurationDays, AreaPreference, Budget, CreatedAt	DurationDays > 0
ItineraryItems	ItineraryItemId	ItineraryId, PlaceId, DayNumber, ItemOrder, Notes	Unique itinerary/day/order

Relationships

- Governorates 1:N Places; Categories 1:N Places with parent/child hierarchy.
- Users M:N Roles through UserRoles; Users M:N Companies through CompanyMembers.
- Companies 1:N Tours; Tours 1:N TourStops and Departures; Places 1:N TourStops.
- Departures 1:N Bookings; Users 1:N Bookings; Bookings reference a specific departure.
- Places M:N Facilities through PlaceFacilities; Places 1:N images and fees.
- Users 1:N Itineraries; Itineraries 1:N Items; users/places linked by Favorites and PlaceReviews.

Normalization and integrity

- Target 3NF: repeated values such as images, fees and facilities are held in related tables.
- Many-to-many relationships use junction tables; sequence/order belongs to TourStops and ItineraryItems.
- Recommended indexes: email, published place filters, tour publication/company, departure date/status, traveler bookings.
- Use check constraints for prices, fees, participant counts, capacity, ratings, dates and coordinates.

17. Booking Rules, Security & API Baseline

Booking integrity: Pending requests do not reserve seats automatically under the proposed baseline. Company confirmation starts a transaction, reads remaining capacity, verifies confirmed participants plus requested participants do not exceed capacity, updates status and commits atomically. Otherwise, return a conflict and keep the request unconfirmed.

Price snapshot: Store the unit and total price when the request is submitted. Later edits to a tour price must not rewrite the historical booking total.

Status lifecycle: Pending may transition to Confirmed, Rejected or Cancelled; Confirmed may transition to Cancelled only when policy allows. Rejected and Cancelled are terminal for the MVP unless an approved rule says otherwise.

Security: hash passwords; apply role and company ownership checks on the server; use environment variables or a secret manager for credentials; validate every request; avoid exposing internal exception details; restrict database network access.

AI boundary: the planner must retrieve candidate places from published database records. The AI must not invent destinations, costs, opening times or live tour availability. Provider failure must not disable core browsing or booking.

Method	Route	Purpose	Access
POST	/api/auth/register	Create traveler account	Public
POST	/api/auth/login	Authenticate	Public
GET	/api/places	Search/filter published places	Public
GET	/api/places/{id}	Place details	Public
GET	/api/tours	Browse published tours	Public
GET	/api/tours/{id}	Tour details and departures	Public
POST	/api/bookings	Create Pending request	Traveler
GET	/api/bookings/my	My bookings	Traveler
PATCH	/api/company/bookings/{id}/decision	Accept/reject request	Authorized company
POST	/api/planner/itineraries/generate	Generate itinerary draft	Policy-defined
POST	/api/itineraries	Save itinerary	Traveler
POST/PATCH	/api/admin/places	Create/update place	Admin

18. Testing Plan & Open Design Decisions

ID	Scenario	Expected result	Requirement mapping
TC-01	Browse places and open details	Correct published record and details	FR-05, FR-09
TC-02	Search and combine filters	Only matching/published records	FR-06 to FR-08
TC-03	Unapproved company publishes tour	API blocks publish; approved owner can manage own tour	FR-14 to FR-16
TC-04	Tour stops and departures	Order is preserved; invalid dates/capacity rejected	FR-18, FR-19
TC-05	Traveler submits booking	Login; Pending status; participant and price snapshots	FR-21, FR-22, FR-24
TC-06	Company confirms booking	Ownership checked; insufficient capacity rejected	FR-23, FR-25, FR-26
TC-07	Cross-user/company access	403/404; private data is not leaked	FR-03, NFR-02, NFR-04
TC-08	AI planner and provider outage	Only stored places recommended; core app remains available	FR-28 to FR-32
TC-09	Fees/facilities missing	Unknown values remain unavailable, not invented	FR-11, FR-12
TC-10	Clean teammate setup	README and seed data allow local execution	NFR-09, NFR-14

Open decisions to confirm

Decision	Current baseline / action
Frontend framework	Select Angular or HTML/CSS/JavaScript based on team delivery ability; update README.
Company membership policy	Confirm whether one or multiple representatives can manage a company; model supports members.
Pending seat policy	Confirm that Pending does not hold seats; capacity check at company confirmation is the proposed rule.
Cancellation policy	Define when traveler/company may cancel a confirmed booking and whether seats are released.
AI provider	Select only after place APIs are stable; keep a deterministic fallback.
Deployment	Select host, domain, backups and CI/CD; current diagram is conceptual.
MVP feature priority	Protect browse → tour → booking; defer Should/Could features if behind schedule.

Conclusion: The V2 ERD and layered modular architecture form a practical design baseline. The most critical integrity rule is that each booking references a specific departure, retains a price snapshot and is confirmed only after a safe capacity check. Team and instructor review is required for the open decisions above.